

I# Relatório — Lab 5 (MC833): Redirecionando o endereço de retorno para dentro de `input`

Por que não basta sobrescrever o retorno com qualquer endereço

A seção anterior ([01-analise-stack-gdb-gef.md](#)) mostrou que os bytes 72–79 do payload caem exatamente sobre o **endereço de retorno salvo** no frame de `bof`. Só sobrescrever esse endereço não é suficiente — é preciso que ele aponte para um lugar na memória onde exista *código executável útil* (o nosso shellcode).

O candidato óbvio seria apontar de volta para dentro do próprio `buffer` (onde, supostamente, o payload também estaria). O problema é que `strcpy` para de copiar no primeiro byte `\x00` que encontrar na origem (`str`). O shellcode gerado por `create_hex_command()` contém bytes `\x00` no meio (por exemplo, nas instruções `mov rax, imm64` cujos imediatos têm bytes altos zerados, como em `0x632d` para `"-c\0"`). Isso significa que, se o shellcode estivesse posicionado nos bytes que o `strcpy` efetivamente copia, ele seria **truncado** no meio — o `strcpy` simplesmente pararia de escrever ao encontrar o primeiro `\x00` do shellcode, e o restante do buffer de destino nunca seria preenchido com o que vem depois.

A saída foi não depender do `strcpy` para colocar o shellcode em memória. O shellcode fica em `input[]`, dentro do frame de `main` — array que foi preenchido **diretamente pelo `read(STDIN_FILENO, input, 500)`**, não pelo `strcpy`. Diferença crucial: `read()` copia exatamente os `n` bytes pedidos, byte a byte, **sem se importar com `\x00`** — não é uma função de string, não tem conceito de terminador. Então o shellcode inteiro (com seus `\x00` internos) chega intacto em `input`, mesmo que o mesmo conteúdo, se copiado via `strcpy`, fosse cortado no meio.

Ou seja: usamos o `strcpy` só para o que ele *precisa* fazer (sobrescrever os 8 bytes do endereço de retorno, que ficam bem no início do payload, antes de qualquer `\x00`) e usamos o `read()` — que já rodou antes, no `main`, e não tem essa limitação — para colocar o shellcode de forma confiável em `input`. O retorno é então redirecionado para dentro de `input`, não para dentro de `buffer`.

O código do offset (`getFile()`, em `shellcode/main.py`)

```
def getFile(bufaddr, shellcode):
    """
    Layout do payload (500 bytes, = tamanho de `input` em main):

        [0:72)      preenchimento (NOP) -- nunca é executado, só precisa nao
        ter \x00
        [72:80)     endereco de retorno -- aponta para dentro de `input`
        (main),
                    onde o NOP sled + shellcode ficam intactos (o read() nao
        trunca em \0,
                    diferente do strcpy que soh alcanca ateh aqui)
        [80:100)    NOP sled
        [100:...)   shellcode (create_hex_command, sem alteracao)
    """
    OFFSET = 72 # distancia confirmada via GDB:
    buffer -> retorno
    INPUT_OFFSET_FROM_BUFFER = 0x50 # distancia fixa entre buffer (bof)
    e input (main)
    SHELL_START = 100
```

```

content = bytearray(0x90 for _ in range(500))
content[SHELL_START:SHELL_START + len(shellcode)] = shellcode

input_addr = bufaddr + INPUT_OFFSET_FROM_BUFFER
ret = input_addr + 80 # aponta pro começo do NOP sled,
dentro de `input`

content[OFFSET:OFFSET + 8] = ret.to_bytes(8, byteorder="little")

return content

```

O que cada constante representa

- **OFFSET = 72** — a distância, em bytes, do início de **buffer** até o endereço de retorno salvo na pilha de **bof**. Confirmada estaticamente (via **disassemble**) e depois experimentalmente (payloads de teste na seção 01): **0x48 = 72**.
- **INPUT_OFFSET_FROM_BUFFER = 0x50** — a distância fixa, em bytes, entre o endereço de **buffer** (dentro do frame de **bof**) e o endereço de **input** (dentro do frame de **main**). Como os dois frames têm layout fixo (mesmo binário, sem ASLR entre **buffer** e **input** — só a base do processo varia, não a distância relativa entre variáveis locais), essa distância também foi medida ao vivo no GDB comparando os dois endereços, e é constante entre execuções.
- **SHELL_START = 100** — onde, dentro dos 500 bytes do payload, o shellcode de fato começa. Entre o byte 80 (logo depois do endereço de retorno) e o byte 100 fica um *NOP sled* de 20 bytes (**0x90**), uma margem de segurança para o salto não precisar ser cirurgicamente exato.
- **bufaddr** — não é uma constante; é o endereço real de **buffer** **vazado pelo próprio servidor** a cada conexão (o **printf("[INFO LEAK] ...")** em **server.c**). O exploit lê esse valor de uma conexão de sondagem antes de mandar o payload de verdade, e usa-o para calcular os endereços abaixo dinamicamente.

O cálculo do novo endereço de retorno

```

input_addr = bufaddr + INPUT_OFFSET_FROM_BUFFER # endereço de `input`,
calculado a partir do leak
ret = input_addr + 80 # aponta pro início do
NOP sled dentro de `input`
content[OFFSET:OFFSET + 8] = ret.to_bytes(8, byteorder="little")

```

Passo a passo:

1. **bufaddr** (vazado pelo servidor) dá o endereço real de **buffer** **nesta execução específica** do processo.
2. Somar **INPUT_OFFSET_FROM_BUFFER (0x50)** move esse endereço do frame de **bof** para o frame de **main** — chegando ao endereço real de **input**.

3. Somar mais **80** desloca do início de **input** até o **início do NOP sled** (byte 80 do payload — logo depois de onde o endereço de retorno foi escrito). É esse ponto, e não o início de **input**, que queremos executar primeiro.
4. **ret.to_bytes(8, byteorder="little")** converte o endereço de 64 bits para os 8 bytes que o **retq** de **bof** vai efetivamente ler da pilha — em **little-endian**, porque é assim que a arquitetura x86-64 armazena valores multi-byte na memória (byte menos significativo no endereço mais baixo).
5. Esses 8 bytes substituem exatamente **content[72:80]** — a posição que a seção 01 confirmou ser o endereço de retorno salvo.

Quando **bof** executa seu **retq** final, ele não volta para **main** no ponto normal de retorno — ele "pula" para dentro de **input**, cai bem no início do NOP sled, e desliza (**0x90 = nop**, instrução que não faz nada além de avançar o **rip**) até o shellcode de verdade, que então executa com os privilégios do processo do servidor.